



**Universidad
Tecnológica
de Bolívar**

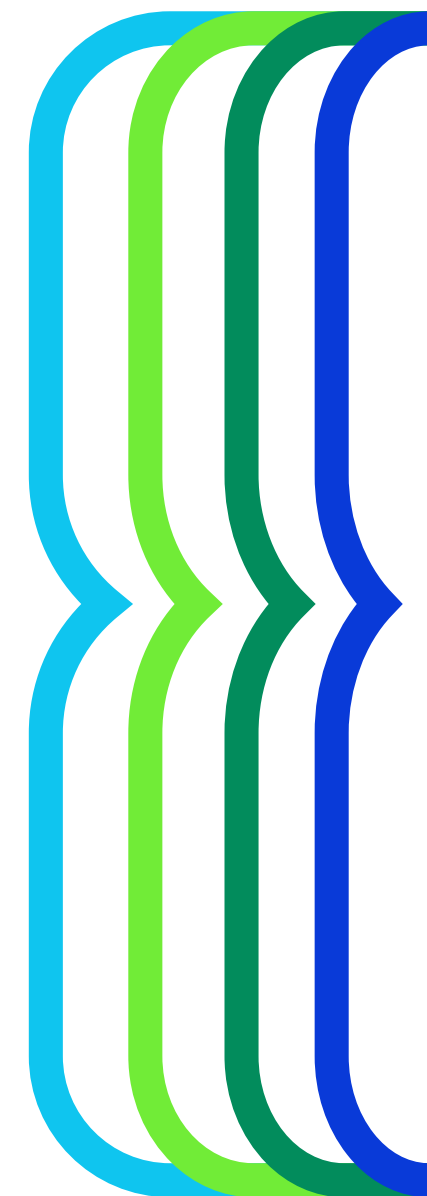
Proyecto de Matemáticas Avanzadas: Modelado y solución de ecuaciones diferenciales ordinarias lineales de segundo orden

Integrantes:

Daniel Andres Arias Lopez
Carlos Daniel Agamez Palomino
Harold Styven Lagares De Voz

Profesor: Enrique Pereira Batista

Matemáticas Avanzadas 2026-01
Maestría en Ingeniería
Universidad Tecnológica de Bolívar



Sistema Masa-Reso

Solución Analítica

Partimos de la ecuación diferencial de segundo orden que modela nuestro sistema:

$$mx'' + cx' + kx = 0 \quad (2)$$

Buscamos soluciones de la forma $x(t) = e^{rt}$. Al sustituir en la ecuación (2), obtenemos la ecuación característica:

$$mr^2 + cr + k = 0 \quad (4)$$

Las raíces de esta ecuación están dadas por la fórmula cuadrática:

$$r_{1,2} = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m} \quad (5)$$

Solución Analítica

Dependiendo del discriminante ($c^2 - 4mk$), existen tres casos teóricos. Asumiremos el caso más representativo en ingeniería para sistemas oscilantes, que es el **sistema subamortiguado** ($c^2 < 4mk$). En este caso, las raíces son complejas conjugadas: $r_{1,2} = -\alpha \pm i\beta$, donde $\alpha = \frac{c}{2m}$ y $\beta = \frac{\sqrt{4mk - c^2}}{2m}$.

La solución analítica general es:

$$x(t) = e^{-\alpha t} (C_1 \cos(\beta t) + C_2 \sin(\beta t)) \quad (6)$$

Aplicando las condiciones iniciales:

1. Evaluando en $t = 0$: $x(0) = e^0(C_1(1) + C_2(0)) \implies C_1 = x_0$
2. Derivando $x(t)$ y evaluando en $t = 0$: $x'(0) = -\alpha C_1 + \beta C_2 = v_0 \implies C_2 = \frac{v_0 + \alpha x_0}{\beta}$

Por lo tanto, la solución analítica final es:

$$x(t) = e^{-\alpha t} \left(x_0 \cos(\beta t) + \frac{v_0 + \alpha x_0}{\beta} \sin(\beta t) \right) \quad (7)$$

Solución Analítica

```
# Librerías
import numpy as np
import matplotlib.pyplot as plt

# Parámetros del sistema
m = 1.0    # Masa
k = 10.0   # Constante del resorte
c = 0.8    # Coeficiente de amortiguamiento
x0 = 0.1   # Posición inicial
v0 = 0.0   # Velocidad inicial
T = 10     # Tiempo total de simulación

# Definición de la solución analítica
def solucion_analitica(t, m, c, k, x0, v0):
    alpha = c / (2 * m)
    omega_d = np.sqrt(4 * m * k - c**2) / (2 * m)

    A = x0
    B = (v0 + alpha * x0) / omega_d

    solution = np.exp(-alpha * t) * (A * np.cos(omega_d * t)
+ B * np.sin(omega_d * t))

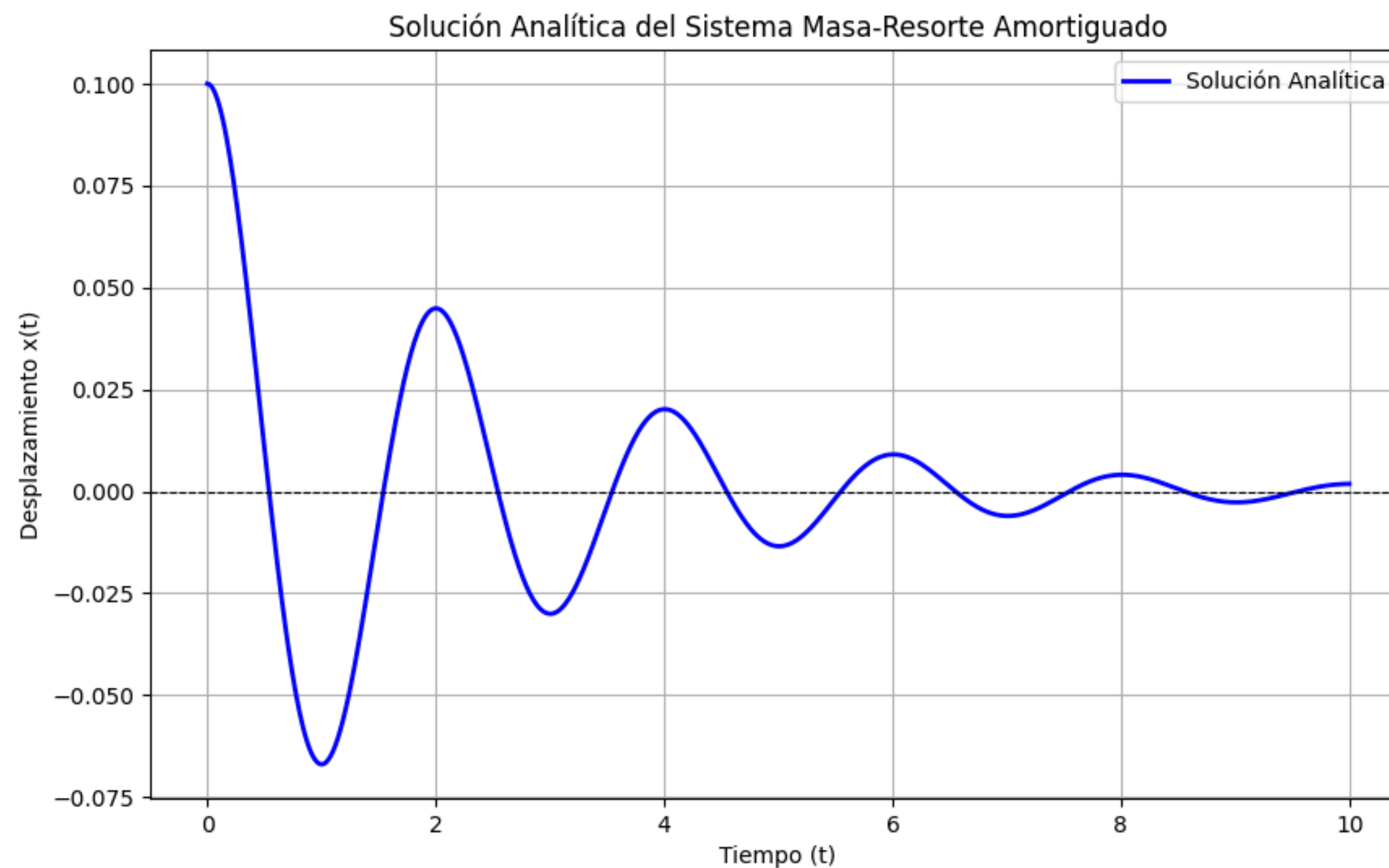
    return solution
```

Solución Analítica

```
# Generar datos
t = np.linspace(0, T, 1000)
x_t = solucion_analitica(t, m, c, k, x0, v0)

# Graficar
plt.figure(figsize=(10, 6))
plt.plot(t, x_t, 'b', linewidth=2, label=
'Solución Analítica')
plt.axhline(0, color='black', linewidth=0.8, linestyle=
'--')
plt.xlabel('Tiempo (t)')
plt.ylabel('Desplazamiento x(t)')
plt.title(
'Solución Analítica del Sistema Masa-Resorte Amortiguado')
plt.legend()
plt.grid(True)
plt.show()
```

Solución Analítica



Solución Numérica (Forma Matricial/Vectorial)

Partimos de la ecuación diferencial de segundo orden que modela nuestro sistema:

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = 0 \quad (1)$$

Paso a paso:

Para lograr esta conversión, creamos dos nuevas variables que representan los estados físicos del sistema:

- $x_1 = x(t)$ (La posición)
- $x_2 = \dot{x}(t)$ (La velocidad)

.....

Solución Numérica (Forma Matricial/Vectorial)

1. **Primera ecuación (Derivada de la posición):** La derivada de la posición (x_1) es simplemente la velocidad (x_2). Expresado matemáticamente y ordenando las variables, queda:

$$\dot{x}_1 = 0 \cdot x_1 + 1 \cdot x_2$$

2. **Segunda ecuación (Derivada de la velocidad):** La derivada de la velocidad ($\dot{x}_2$) es la aceleración ($\ddot{x}$). Para hallarla, despejamos $\ddot{x}$ de nuestra ecuación original (1) y reemplazamos con nuestras nuevas variables:

$$\ddot{x} = -\frac{k}{m}x(t) - \frac{c}{m}\dot{x}(t) \quad \Longrightarrow \quad \dot{x}_2 = -\frac{k}{m}x_1 - \frac{c}{m}x_2$$

Construcción de la matriz A y el vector Y:

Si agrupamos nuestras variables en un vector columna llamado $\mathbf{y}(t) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, podemos reescribir las dos ecuaciones anteriores de forma matricial:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Escrito de forma compacta:

$$\dot{\mathbf{y}}(t) = \mathbf{A}\mathbf{y}(t)$$

Solución Numérica (Forma Matricial/Vectorial)

Solución general:

La ventaja de tener el sistema en la forma $\dot{\mathbf{y}}(t) = \mathbf{A}\mathbf{y}(t)$ es que su solución analítica exacta está dada directamente por la **matriz exponencial**:

$$\mathbf{y}(t) = e^{\mathbf{A}t}\mathbf{y}_0 \quad (2)$$

Donde $\mathbf{y}_0$ es el vector con la posición y velocidad iniciales $\begin{bmatrix} x(0) \\ v(0) \end{bmatrix}$. Al multiplicar la matriz exponencial $e^{\mathbf{A}t}$ por este vector, el sistema proyecta ambos valores hacia el futuro en cualquier instante t .

¿Qué graficamos en esta solución?

Al resolver $\mathbf{y}(t) = e^{\mathbf{A}t}\mathbf{y}_0$ calculamos un vector para cada instante de tiempo. Estaremos graficando:

- **El primer elemento del vector resultante (x_1):** Es la posición de la masa a lo largo del tiempo. Veremos su oscilación y cómo la fricción reduce su amplitud.
- **El segundo elemento del vector resultante (x_2):** Es la velocidad de la masa, que oscila desfasada de la posición hasta detenerse en cero.

Solución Numérica (Forma Matricial/Vectorial)

Implementación de uso general en Python

Este código en Python utiliza la función `expm` de `SciPy` para calcular e^{At} exactamente como se define en la teoría matricial.

Listing 1: Solución y gráfica del sistema matricial usando Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import expm

# 1. Definición de los parametros fisicos
m = 1.0    # Masa [kg]
k = 10.0   # Constante del resorte [N/m]
c = 0.8    # Coeficiente de amortiguamiento viscoso [N*s/m]

# 2. Condiciones iniciales
x0 = 0.1   # Posicion inicial [m]
v0 = 0.0   # Velocidad inicial [m/s]
y0 = np.array([x0, v0]) # Vector y_0 = [x1, x2]

# 3. Construcción de la Matriz A del sistema
# Fila 1: [0, 1] -> la derivada de posicion es velocidad
# Fila 2: [-k/m, -c/m] -> la ecuacion de aceleracion
A = np.array([[ 0.0,    1.0],
               [-k/m,   -c/m]])

# 4. Vector de tiempo (10 segundos, 500 puntos)
t = np.linspace(0, 10, 500)
```

Solución Numérica (Forma Matricial/Vectorial)

```
posicion = np.zeros(len(t))
velocidad = np.zeros(len(t))

# 5. Calculo iterativo con la matriz exponencial
for i, tiempo in enumerate(t):
    # Calcula e^(A*t)
    matriz_exp = expm(A * tiempo)

    # Resuelve y(t) = e^(At) * y0
    y_actual = np.dot(matriz_exp, y0)

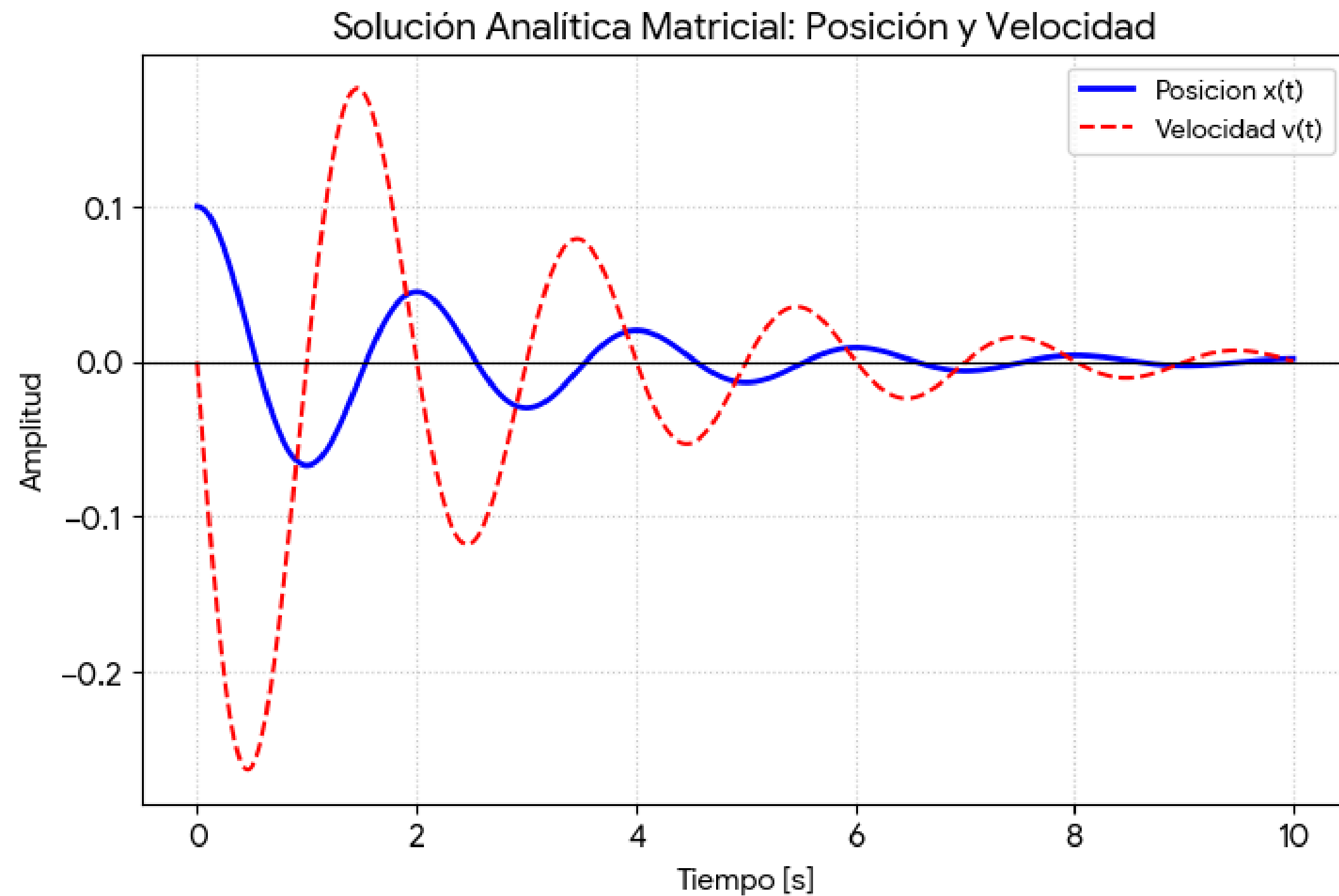
    # Extrae x1 (posicion) y x2 (velocidad)
    posicion[i] = y_actual[0]
    velocidad[i] = y_actual[1]

# 6. Graficacion
plt.figure(figsize=(8, 5))
plt.plot(t, posicion, label='Posicion x(t)', color='blue')
plt.plot(t, velocidad, label='Velocidad v(t)', color='red', linestyle='--')

plt.axhline(0, color='black', linewidth=0.5)
plt.title('Solucion Analitica Matricial: Posicion y Velocidad')
plt.xlabel('Tiempo [s]')
plt.ylabel('Amplitud')
plt.legend()
plt.grid(True)

plt.show()
```

Solución Numérica (Forma Matricial/Vectorial)



Solución Numérica (Forma Escalar)

Para resolver la EDO usando el método de Euler, aplicamos una discretización en diferencias finitas hacia adelante. Reducimos la ecuación de segundo orden a dos ecuaciones de primer orden introduciendo la variable de velocidad $v(t) = x'(t)$:

$$x'(t) = v(t) \quad (8)$$

$$v'(t) = -\frac{k}{m}x(t) - \frac{c}{m}v(t) \quad (9)$$

Aplicamos diferencias finitas $y'(t) \approx \frac{y_{n+1} - y_n}{h}$ con un tamaño de paso h :

$$\frac{x_{n+1} - x_n}{h} = v_n \quad (10)$$

$$\frac{v_{n+1} - v_n}{h} = -\frac{k}{m}x_n - \frac{c}{m}v_n \quad (11)$$

Despejando, obtenemos el **método de Euler explícito en su forma escalar**:

$$x_{n+1} = x_n + h \cdot v_n \quad (12)$$

$$v_{n+1} = v_n + h \left(-\frac{k}{m}x_n - \frac{c}{m}v_n \right) \quad (13)$$

Solución Numérica (Forma Escalar)

```
# Librerías
import numpy as np
import matplotlib.pyplot as plt

# Parámetros definidos en el proyecto
m = 1.0
k = 10.0
c = 0.8
x0 = 0.1
v0 = 0.0
T = 10
h = 0.05 # Tamaño de paso

# Preparación de arreglos
N = int(T / h)
t = np.linspace(0, T, N + 1)
x_num = np.zeros(N + 1)
v_num = np.zeros(N + 1)

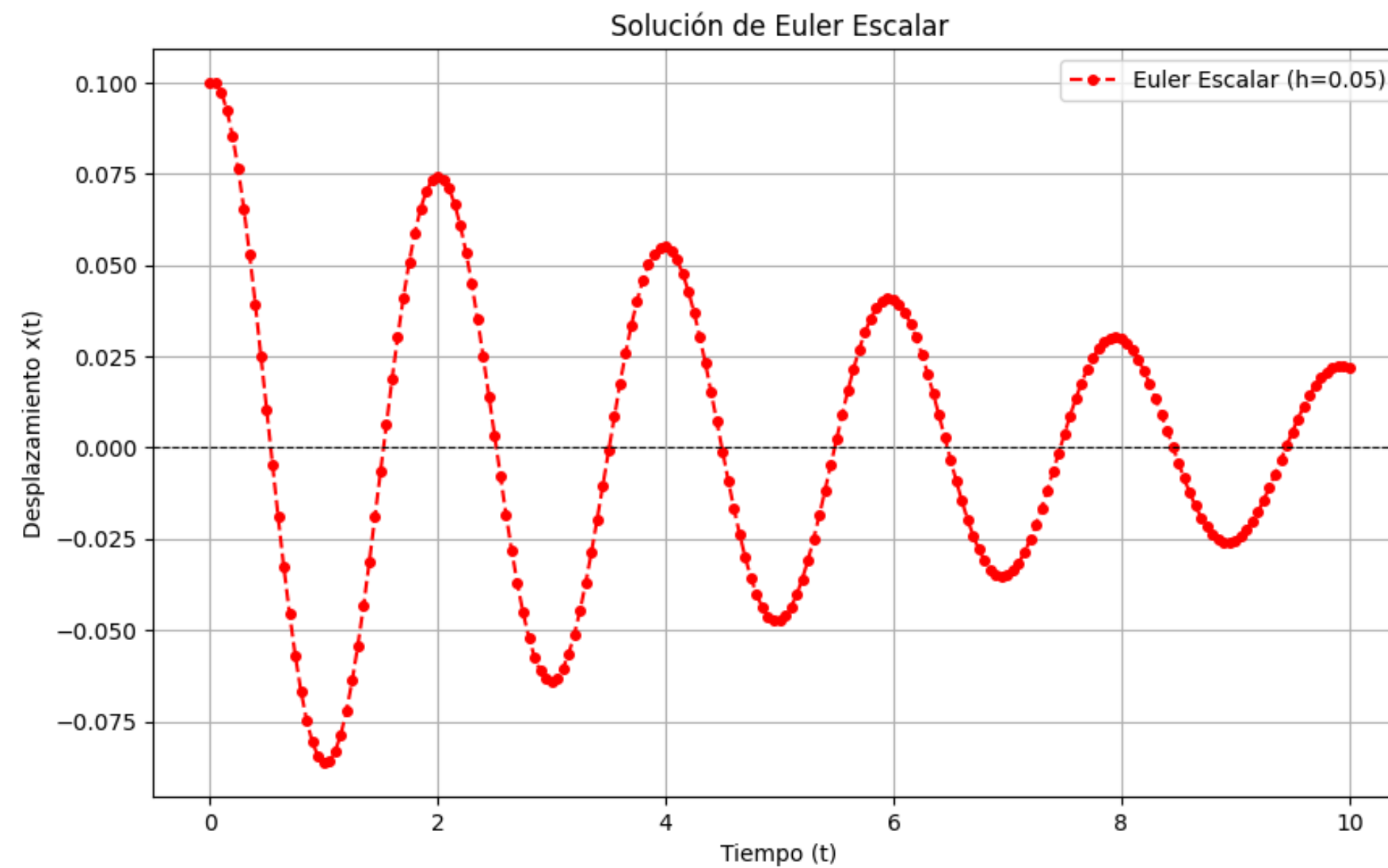
# Condiciones iniciales
x_num[0] = x0
v_num[0] = v0
```

Solución Numérica (Forma Escalar)

```
# Bucle de iteración: Método de Euler Escalar
for n in range(N):
    x_num[n+1] = x_num[n] + h * v_num[n]
    v_num[n+1] = v_num[n] + h * (-(c/m)*v_num[n] - (k/m)*x_num[n])

# Graficación de resultados
plt.figure(figsize=(10, 6))
plt.plot(t, x_num, 'r--', marker='o', markersize=4, label=f
'Euler Escalar (h={h})')
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.xlabel('Tiempo (t)')
plt.ylabel('Desplazamiento x(t)')
plt.title('Solución de Euler Escalar')
plt.legend()
plt.grid(True)
plt.show()
```

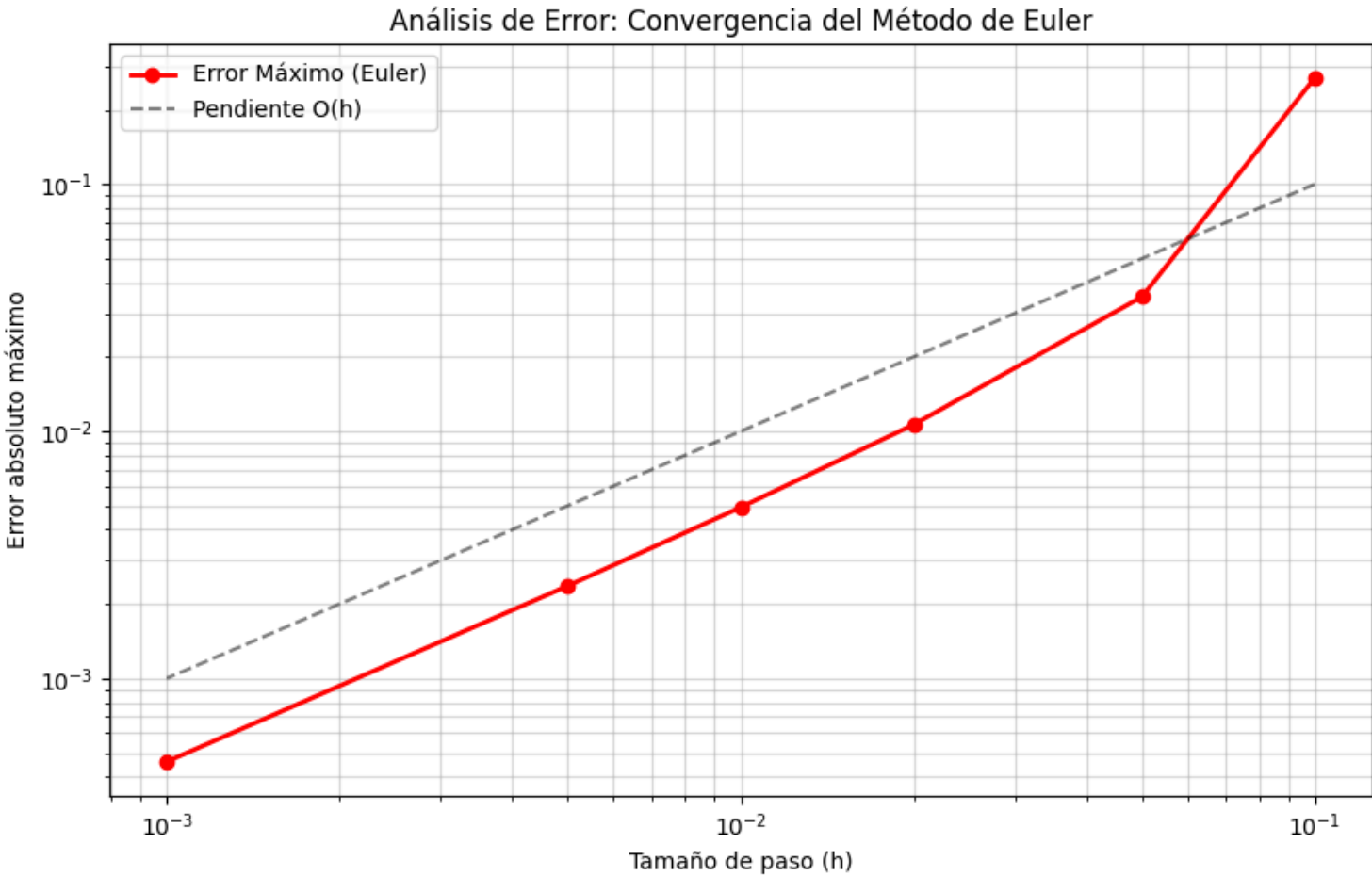
Solución Numérica (Forma Escalar)



Análisis Numérico (Errores)

Error absoluto:

$$E_n = |x(t_n) - x_n^{\text{num}}|, \quad E_{\text{máx}} = \max_n E_n.$$



Paso (n)	Tiempo (t)	x Analítica	x Euler	Error Absoluto
0	0.00	0.100000	0.100000	0.000000
1	0.10	0.095171	0.100000	0.004829
2	0.20	0.081643	0.090000	0.008357
3	0.30	0.061374	0.070800	0.009426
4	0.40	0.036813	0.044136	0.007323
5	0.50	0.010633	0.012525	0.001892
6	0.60	-0.014548	-0.020970	0.006423
7	0.70	-0.036406	-0.053039	0.016633
8	0.80	-0.053114	-0.080445	0.027330
9	0.90	-0.063476	-0.100354	0.036879
10	1.00	-0.066991	-0.110627	0.043636

Análisis de Resultados

A partir de las gráficas y los datos obtenidos en las distintas implementaciones, se destacan los siguientes comportamientos correspondientes a los tres modelos presentados:

- **Modelo Analítico:** Proporciona la solución exacta del sistema en el régimen subamortiguado. Demuestra el decaimiento exponencial característico de la amplitud debido al término de amortiguamiento viscoso y mantiene una frecuencia de oscilación constante ω_d . Este modelo sirve como la referencia absoluta (*ground truth*) para cuantificar la precisión de las aproximaciones numéricas.
- **Modelo Numérico (Euler Escalar):** Permite aproximar el comportamiento dinámico del sistema discretizando iterativamente la posición y la velocidad. Logra seguir la tendencia oscilatoria amortiguada de manera aceptable, pero al ser un método explícito de primer orden, está sujeto a errores de truncamiento proporcionales al tamaño de paso h ($\mathcal{O}(h)$), lo que se evidencia en un ligero desfase temporal o alteración artificial de la amplitud si h no es suficientemente pequeño.
- **Modelo Numérico (Euler Vectorial o Matricial):** Produce resultados numéricos exactamente idénticos al enfoque escalar al tratarse del mismo algoritmo subyacente de Euler hacia adelante. No obstante, al estructurarse como un modelo de espacio de estados, ofrece una formulación matemática mucho más compacta, escalable a sistemas de múltiples grados de libertad y computacionalmente eficiente para su implementación iterativa en software como Matlab.

Análisis de Resultados

Para facilitar la decisión sobre qué aproximación emplear según el contexto del problema, la Tabla 1 presenta una comparativa de los escenarios de uso recomendados para cada modelo.

Cuadro 1: Comparativa de los modelos y escenarios de uso recomendados			
Modelo	Cuándo usarlo (Escenario ideal)	Observaciones principales	
Analítico	Sistemas lineales simples (1 grado de libertad). Validación rigurosa de algoritmos numéricos.	Brinda precisión absoluta, pero es inviable matemáticamente en sistemas complejos o no lineales.	
Euler Escalar	Fines puramente didácticos o cuando se requiere programar en lenguajes de bajo nivel sin librerías de álgebra lineal.	Fácil de programar e interpretar paso a paso; tedioso de escalar a más de un grado de libertad.	
Euler Vectorial	Sistemas de múltiples grados de libertad. Implementaciones en lenguajes optimizados (Matlab, Python) y aplicaciones de control moderno.	Formulación matemática elegante y escalable. Aprovecha la eficiencia computacional de las operaciones matriciales.	



**Universidad
Tecnológica
de Bolívar**

¡GRACIAS!

www.utb.edu.co |        **utboficial**

Copyright © 2023. Todos los derechos reservados Universidad Tecnológica de Bolívar.